

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И
РАДИОЭЛЕКТРОНИКИ»
Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

ОТЧЕТ
по лабораторной работе № 1
по дисциплине «Методы машинного обучения»
на тему: «Основы языка Python, алгоритмы обработки данных,
обработка исключений и объектно-ориентированное программирование»
Вариант № 7

Выполнил(а):
студент(ка) гр. 351002
Валинский А. С.

Проверил:
преподаватель
Лебедев А. В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью выполнения лабораторной работы является:

1. Освоение базового синтаксиса языка Python, концепции динамической типизации и встроенных структур данных (числовые типы, строки, списки, кортежи, словари), а также алгоритмов их модификации, фильтрации и сортировки.
2. Изучение модульного построения программ с помощью функций, механизмов передачи параметров, возврата значений и гарантированной обработки исключительных ситуаций конструкцией `try..except..else..finally`.
3. Получение практических навыков реализации объектно-ориентированной модели: проектирование классов, разграничение доступа к атрибутам с помощью декораторов свойств (`@property`), реализация механизма наследования, полиморфизма и специальных методов.
4. Разработка собственной динамической структуры данных (односвязный список) и организация долговременного хранения данных посредством файлового ввода-вывода.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Динамическая типизация и встроенные типы данных Python

Одной из ключевых архитектурных черт языка Python является сильная динамическая типизация. Переменная в Python не требует предварительного объявления типа: имя переменной выступает в роли ссылки (указателя) на созданный в динамической памяти объект. Тип данных связывается непосредственно с объектом в момент присваивания значения.

Стандартная библиотека языка содержит широкий спектр скалярных и составных типов данных:

- **int** и `\texttt{float}` — целые числа произвольной разрядности и числа с плавающей запятой двойной точности.
- **bool** — логический тип, принимающий значения `True` или `False` и являющийся подклассом целых чисел.
- **str** — неизменяемая последовательность символов в кодировке Unicode. Поддерживает операции индексирования, срезов (*slicing* вида `[start:stop:step]`), а также специализированные методы проверки регистра (`.isupper()`, `.islower()`), поиска подстрок и конкатенации.
- **list** — динамический изменяемый гетерогенный массив, обеспечивающий добавление в конец за амортизированное время $O(1)$ и произвольный доступ по индексу.
- **tuple** — неизменяемый кортеж фиксированной длины, потребляющий меньше системных ресурсов и гарантирующий защиту элементов от случайной модификации.
- **dict** — ассоциативный массив пар «ключ–значение», функционирующий на основе хеш-таблиц. Обеспечивает среднюю временную сложность поиска, вставки и удаления за $O(1)$. Ключами словаря могут выступать только хешируемые неизменяемые типы данных.

Сортировка последовательностей в Python реализуется стандартной функцией `sorted()` на базе адаптивного алгоритма Timsort с временной сложностью в худшем и среднем случаях $O(n \log n)$. Порядок сортировки и критерий сравнения регулируются именованными аргументами `reverse` и `key`.

2.2 Функции и механизм обработки исключений

Функции представляют собой самостоятельные именованные блоки инструкций, позволяющие устранить дублирование логики и декомпозировать сложную вычислительную задачу на элементарные подзадачи. Объявление функции выполняется ключевым словом `def`, за которым следует сигнатура аргументов и тело с отступом в четыре пробела.

Если инструкция `return` отсутствует, функция возвращает специальный объект `None`.

Исключения (*exceptions*) в Python представляют собой специализированный тип данных, служащий для сигнализации об ошибках времени выполнения и аномальных состояниях программы. Для предотвращения аварийного завершения процесса применяется блочная конструкция:

```
try:
    # Контролируемый блок инструкций
    result = risky_operation()
except SpecificException as error:
    # Перехват и нейтрализация конкретного типа ошибки
    handle_error(error)
else:
    # Выполняется только при отсутствии исключений в блоке t
ry
    process_success(result)
finally:
    # Гарантированно исполняется при любом исходе
    cleanup_resources()
```

Иерархия исключений базируется на суперклассе `BaseException` и его основном производном классе `Exception`. Среди наиболее распространенных стандартных ошибок выделяются:

- **TypeError** — передача аргумента неподдерживаемого типа;
- **ValueError** — передача аргумента корректного типа, но недопустимого значения;
- **IndexError** — обращение к элементу по индексу, выходящему за границы последовательности;
- **ZeroDivisionError** — попытка деления числа на ноль.

2.3 Объектно-ориентированное программирование в Python

Объектно-ориентированный подход структурирует программный комплекс в виде взаимодействующих сущностей (объектов), являющихся экземплярами классов. Определение класса задается директивой `class`. В соответствии с соглашением PEP-8 имена классов записываются в стиле CamelCase.

Ключевые концепции ООП в Python реализуются следующим образом:

1. **Инициализатор** `__init__` и ссылка `\texttt{self}`: метод `__init__` вызывается непосредственно после создания экземпляра методом `__new__` для инициализации атрибутов объекта. Параметр `self` связывается с конкретным экземпляром и обеспечивает доступ к его внутреннему состоянию.
2. **Инкапсуляция**: сокрытие деталей внутренней реализации и валидация данных осуществляются соглашением о защищенных полях (префикс

подчеркивания `_name`) и связкой декораторов `@property` (геттер) и `@.setter` (сеттер).

3. **Наследование:** производный класс расширяет интерфейс базового класса, а инициализация родительских атрибутов производится вызовом `super().__init__(...)`.
4. **Специальные методы (dunder-методы):** методы `__str__()` и `__len__()` интегрируют пользовательские объекты с системными функциями `print()`, `str()` и `len()`.

2.4 Реализация динамических структур данных на базе классов

Линейный односвязный список представляет собой динамическую цепочку независимых узловых элементов. Каждый узел инкапсулирует информационное поле `value` и адресную ссылку на следующий узел `nextElem`:

$$Node_i = \langle value_i, \&Node_{i+1} \rangle, \quad Node_n.nextElem = None. \quad (1)$$

В отличие от стандартного вектора, добавление и удаление элементов в голову списка выполняется за время $O(1)$ без выделения непрерывного блока памяти и перемещения соседних элементов. Доступ к произвольному k -му элементу осуществляется последовательным обходом ссылок со сложностью $O(k)$.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Согласно варианту № 7 индивидуальное задание лабораторной работы состоит из трех взаимосвязанных частей:

3.1 Часть 1. Базовые алгоритмы и структуры данных

1. **Задание 1.** Найти сумму нечетных цифр введенного с клавиатуры натурального числа.
2. **Задание 2.** Посчитать, сколько пар стоящих рядом символов верхнего и нижнего регистра находится в введенном слове, а также определить общее количество согласных букв.
3. **Задание 3.** Дан гетерогенный список [1612, 49, 'hello', 6, 19, 'world']. Все числа списка проверить на четность: если число четное — вычислить сумму его цифр, если нечетное — заменить его значением 1. Для всех строковых элементов посчитать количество гласных и согласных букв.
4. **Задание 4.** Отсортировать произвольный словарь по значениям в порядке возрастания и убывания.
5. **Задание 5.** Разработать консольное меню «Кондитерская», включающее просмотр описания, цены, остатка на складе, полной сводки, покупку товара с пересчетом баланса и выход.
6. **Задание 6.** Дан кортеж целых чисел. Отсортировать его элементы по убыванию и вывести экстремальные значения (минимальное и максимальное).

3.2 Часть 2. Функции и обработка исключений

Все задания выполнить с обязательным использованием функций и отказоустойчивой обработкой потенциальных исключений блоками `try..except..finally`:

1. **Задание 1.** Разработать функцию `is_password_good(password)`, возвращающую `True`, если длина пароля составляет не менее 10 символов и он одновременно содержит заглавные и строчные буквы.
2. **Задание 2.** Разработать функцию `diffTypeFunction(a)`, принимающую аргумент переменного типа (кортеж — длины элементов; список — сумма после первого отрицательного; число — число четных цифр; строка — сумма чисел). Протестировать все ветки.
3. **Задание 3.** Дана матрица и индексы i и j . Реализовать перестановку столбцов с номерами i и j с проверкой выхода индексов за пределы размерности.
4. **Задание 4.** Реализовать программу, комплексно иллюстрирующую функционирование конструкции `try..except..else..finally`.

3.3 Часть 3. Объектно-ориентированное программирование

1. **Задание 1.** Создать класс односвязного списка `cheengizsList` и реализовать в нем методы добавления (`append`, `prepend`, `insert`), удаления (`remove`), индексации (`get`, `find`) и обращения (`reverse`).
2. **Задание 2.** Спроектировать базовый класс «Транспортное средство» и производные классы «Самолет», «Поезд», «Автомобиль». Рассчитать время и стоимость поездки по маршруту и определить оптимальные варианты. Обеспечить запись отчета в файл.
3. **Задание 3.** Создать классы «Книга», «Отдел» и «Библиотека». Реализовать операции каталогизации, инкапсуляцию полей и выгрузку иерархии в текстовый файл.

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Ниже приведен полный программный код решения задач лабораторной работы, выполненный на языке Python.

Программное решение Части 1 (базовые типы и алгоритмы)

```
# --- Вспомогательные функции ---
def isVowel(x):
    x = x.lower()
    return x in ('a', 'e', 'i', 'o', 'u')

# --- Задание 1: Сумма нечетных цифр числа ---
s = input("Enter a number: ")
sumDigits = 0
for char in s:
    if char in "13579":
        sumDigits += int(char)
print("Sum of odd digits:", sumDigits)

# --- Задание 2: Подсчет регистровых пар и согласных ---
s = input("enter string: ")
sumUpper = 0
sumLower = 0
consonantAmount = 0 if isVowel(s[0]) else 1

for i in range(1, len(s)):
    if s[i-1].isupper() and s[i].isupper():
        sumUpper += 1
    if s[i-1].islower() and s[i].islower():
        sumLower += 1
    if not isVowel(s[i]):
        consonantAmount += 1

print("end upper =", sumUpper)
print("end lower =", sumLower)
print("consonant amount =", consonantAmount)

# --- Задание 3: Обработка гетерогенного списка ---
data = [1612, 49, 'hello', 6, 19, 'world']
print("original list: ", data)
for i in range(len(data)):
    if type(data[i]) == int:
        if data[i] % 2 == 1:
            data[i] = 1
            continue
        digitSum = sum(int(c) for c in str(data[i]))
        print("sum of digits in", data[i], "is", digitSum)
    elif type(data[i]) == str:
        consonants = sum(1 for c in data[i] if not isVowel(c))
        print("consonant amount in", data[i], "is", consonants)
        print("vowel amount in", data[i], "is", len(data[i]))
```

```

- consonants)

print("affected list: ", data)

# --- Задание 4: Сортировка словаря по значениям ---
someDict = {'a': 1, 'b': 5, 'c': 3, 'd': 2}
print("По возрастанию:", dict(sorted(someDict.items(), key=
lambda item: item[1], reverse=False)))
print("По убыванию: ", dict(sorted(someDict.items(), key=la
mbda item: item[1], reverse=True)))

# --- Задание 5: Консольная программа «Кондитерская» ---
bakery = {
    "Торт": ["Бисквитные коржи, крем, шоколадная глазурь", 1
20, 1500],
    "Пирожное": ["Песочное тесто, заварной крем", 80, 800],
    "Маффин": ["Шоколадное тесто, кусочки шоколада", 60, 60
0],
    "Круассан": ["Слоеное тесто, миндальная начинка", 70, 50
0]
}

# --- Задание 6: Сортировка кортежа ---
numTuple = (2, 8, -1, 7, 0, 3, 15555)
print("Original tuple: ", numTuple)
newNumTuple = sorted(numTuple, reverse=True)
print("Sorted tuple: ", newNumTuple)
print("Min value in tuple: ", newNumTuple[-1])
print("Max value in tuple: ", newNumTuple[0])

```

Программное решение Части 2 (функции и исключения)

```

# --- Задание 1: Проверка надежности пароля ---
def is_password_good(password):
    try:
        if len(password) < 10:
            return False
        if not any(symb.isupper() for symb in password):
            return False
        if not any(symb.islower() for symb in password):
            return False
        return True
    except TypeError as e:
        print("Ошибка: неверный тип аргумента:", e)
        return False
    except Exception as e:
        print("Непредвиденная ошибка:", e)
        return False
    finally:
        print("Проверка пароля завершена.")

print("Результат проверки:", is_password_good('PIUBIUBAFIUe
B'))

# --- Задание 2: Функция с переменным типом аргумента ---

```

```

def diffTypeFunction(a):
    try:
        if type(a) == tuple:
            print("Tuple:")
            for i in a:
                print("len of", i, "is", len(i))
        elif type(a) == list:
            print("List:")
            isNegativeFound = False
            sumAfterNegatvie = 0
            for i in range(len(a)):
                if isNegativeFound:
                    sumAfterNegatvie += a[i]
                elif a[i] < 0:
                    isNegativeFound = True
            print("sum after negative:", sumAfterNegatvie)
        elif type(a) == int:
            numStr = str(a)
            print("amount of even digits:", sum(1 for c in numStr if c in "02468"))
        elif type(a) == str:
            wordNumArr = a.split()
            numberSum = sum(int(w) for w in wordNumArr if w.isnumeric())
            print("sum of numbers in string:", numberSum)
        except TypeError as e:
            print("Ошибка типа данных:", e)
        finally:
            print("Обработка завершена.")

diffTypeFunction(("apple", "banana", "kiwi"))
diffTypeFunction([1, 2, -3, 4, 5])
diffTypeFunction(24681)
diffTypeFunction("some 10 20 words")

# --- Задание 3: Перестановка столбцов матрицы ---
def changeIandJcolumnsInMatrix(matrix, i, j):
    try:
        for row in matrix:
            row[i - 1], row[j - 1] = row[j - 1], row[i - 1]
    except IndexError as e:
        print("Ошибка индекса столбца:", e)
    finally:
        print("Операция перестановки столбцов завершена.")

matrix = [[1, 2, 3], [4, 5, 6]]
print("Исходная матрица:", matrix)
changeIandJcolumnsInMatrix(matrix, 1, 3)
print("Матрица после перестановки:", matrix)

# --- Задание 4: Демонстрация try..except..else..finally ---
def process_data(value):
    try:
        print(f"\nОбработка значения: {repr(value)}")
        num = float(value)
        result = 100 / num
    
```

```

except ValueError as e:
    print(f"    [except ValueError]: Ошибка преобразования
({e})")
except ZeroDivisionError as e:
    print(f"    [except ZeroDivisionError]: Деление на нол
ь ({e})")
else:
    print(f"    [else]: Успешный расчет, результат = {resu
lt:.2f}")
finally:
    print("    [finally]: Блок гарантированного завершения
выполнен.")

for val in ["25", "0", "abc", 10]:
    process_data(val)

```

Программное решение Части 3 (объектно-ориентированное программирование)

```

# --- Задание 1: Класс односвязного списка ---
class cheengizsList:
    value = None
    nextElem = None

    def __init__(self, *elements):
        self.value = None
        self.nextElem = None
        for el in elements:
            self.append(el)

    def append(self, val):
        if self.value is None and self.nextElem is None:
            self.value = val
            return
        curr = self
        while curr.nextElem is not None:
            curr = curr.nextElem
        new_node = cheengizsList()
        new_node.value = val
        curr.nextElem = new_node

    def prepend(self, val):
        if self.value is None and self.nextElem is None:
            self.value = val
            return
        new_node = cheengizsList()
        new_node.value = self.value
        new_node.nextElem = self.nextElem
        self.value = val
        self.nextElem = new_node

    def insert(self, index, val):
        if index < 0 or index > self.size():
            raise IndexError("Индекс выходит за границы спис
ка")

```

```

        if index == 0:
            self.prepend(val)
            return
        curr = self
        for _ in range(index - 1):
            curr = curr.nextElem
        new_node = cheengizsList()
        new_node.value = val
        new_node.nextElem = curr.nextElem
        curr.nextElem = new_node

def remove(self, val):
    if self.value == val:
        if self.nextElem is not None:
            self.value = self.nextElem.value
            self.nextElem = self.nextElem.nextElem
        else:
            self.value = None
        return True
    curr = self
    while curr.nextElem is not None:
        if curr.nextElem.value == val:
            curr.nextElem = curr.nextElem.nextElem
            return True
        curr = curr.nextElem
    raise ValueError(f"{val} не найден")

def get(self, index):
    curr, idx = self, 0
    while curr is not None and curr.value is not None:
        if idx == index:
            return curr.value
        curr = curr.nextElem
        idx += 1
    raise IndexError("Индекс вне диапазона")

def find(self, val):
    curr, idx = self, 0
    while curr is not None and curr.value is not None:
        if curr.value == val:
            return idx
        curr = curr.nextElem
        idx += 1
    return -1

def size(self):
    if self.value is None and self.nextElem is None:
        return 0
    count, curr = 0, self
    while curr is not None and curr.value is not None:
        count += 1
        curr = curr.nextElem
    return count

def reverse(self):
    elements = self.to_list()

```

```

        elements.reverse()
        self.clear()
        for x in elements:
            self.append(x)

    def clear(self):
        self.value = None
        self.nextElem = None

    def to_list(self):
        res, curr = [], self
        while curr is not None and curr.value is not None:
            res.append(curr.value)
            curr = curr.nextElem
        return res

    def __str__(self):
        lst = self.to_list()
        return " -> ".join(map(str, lst)) if lst else "Пусто  
й список"

# --- Задание 2: Иерархия транспортных средств ---
class Vehicle:
    def __init__(self, name, speed, cost_per_km):
        self.name = name
        self.speed = speed
        self.cost_per_km = cost_per_km

    def calculate_time(self, distance):
        return distance / self.speed

    def calculate_cost(self, distance):
        return distance * self.cost_per_km

    def get_trip_info(self, from_city, to_city, distance):
        return {
            "vehicle": self.name,
            "route": f"{from_city} -> {to_city}",
            "distance_km": distance,
            "time_hours": round(self.calculate_time(distance)
e), 2),
            "cost_rub": round(self.calculate_cost(distance),
2)
        }

class Airplane(Vehicle):
    def __init__(self, name="Самолет", speed=800, cost_per_k
m=12.0):
        super().__init__(name, speed, cost_per_km)

class Train(Vehicle):
    def __init__(self, name="Поезд", speed=100, cost_per_km=
3.5):
        super().__init__(name, speed, cost_per_km)

class Car(Vehicle):

```

```

    def __init__(self, name="Автомобиль", speed=90, cost_per
_km=5.5):
        super().__init__(name, speed, cost_per_km)

# --- Задание 3: Каталог библиотеки ---
class Book:
    def __init__(self, title, author, year):
        self._title = title
        self._author = author
        self._year = year

    @property
    def title(self):
        return self._title

    @title.setter
    def title(self, new_title):
        if not new_title:
            raise ValueError("Название не может быть пустым")
        self._title = new_title

    def __str__(self):
        return f"«{self._title}» ({self._author}, {self._year})"

class Department:
    def __init__(self, name):
        self._name = name
        self._books = []

    def add_book(self, book):
        self._books.append(book)

    def remove_book(self, title):
        for i, b in enumerate(self._books):
            if b.title.lower() == title.lower():
                return self._books.pop(i)
        return None

    def change_book_title(self, old_title, new_title):
        for b in self._books:
            if b.title.lower() == old_title.lower():
                b.title = new_title
                return True
        return False

    def __str__(self):
        res = f"Отдел: {self._name} ({len(self._books)} книг)\n"
        for i, b in enumerate(self._books, 1):
            res += f"    {i}. {b}\n"
        return res

class Library:
    def __init__(self, name):

```

```
self._name = name
self._departments = {}

def add_department(self, dept):
    self._departments[dept._name] = dept

def save_to_file(self, filename="library_catalog.txt"):
    with open(filename, "w", encoding="utf-8") as f:
        f.write(f"=== Библиотека: {self._name} ===\n\n")
        for dept in self._departments.values():
            f.write(str(dept) + "\n")
```

5 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Разработанные программные модули были протестированы в интерактивной среде Jupyter Notebook под управлением ядра Python 3.14. Все тесты выполнены успешно с формированием ожидаемых выходных данных.

На рисунке 1 представлены результаты консольного вывода выполнения заданий первой части лабораторной работы.

```
● ● ● Jupyter Kernel - Выполнение Части 1 (Базовые структуры данных)

>>> # Задание 1: Сумма нечетных цифр числа
Enter a number: 1357
Sum of odd digits: 16

>>> # Задание 2: Подсчет регистровых пар и согласных
enter string: Hello World
end upper = 0
end lower = 4
consonant amount = 7

>>> # Задание 3: Преобразование гетерогенного списка
original list: [1612, 49, 'hello', 6, 19, 'world']
sum of digits in 1612 is 10
consonant amount in hello is 3, vowel amount in hello is 2
sum of digits in 6 is 6
consonant amount in world is 4, vowel amount in world is 1
affected list: [1612, 1, 'hello', 6, 1, 'world']

>>> # Задание 4: Сортировка словаря
По возрастанию: {'a': 1, 'd': 2, 'c': 3, 'b': 5}
По убыванию: {'b': 5, 'c': 3, 'd': 2, 'a': 1}

>>> # Задание 6: Сортировка кортежа
Original tuple: (2, 8, -1, 7, 0, 3, 15555)
Sorted tuple: [15555, 8, 7, 3, 2, 0, -1]
Min value in tuple: -1, Max value in tuple: 15555
```

Рисунок 1 – Результаты тестирования алгоритмов обработки базовых структур данных части 1

При выполнении Задания 1 для натурального числа 1357 алгоритм корректно просуммировал все нечетные цифры: $1 + 3 + 5 + 7 = 16$. В Задании 2 для тестовой строки «Hello World» обнаружено 4 пары рядом стоящих символов нижнего регистра («el», «ll», «or», «ld») и 7 согласных букв. В Задании 3 исходный гетерогенный список `[1612, 49, 'hello', 6, 19, 'world']` преобразован в `[1612, 1, 'hello', 6, 1, 'world']` с расчетом суммы цифр четных чисел ($1 + 6 + 1 + 2 = 10$) и подсчетом гласных/согласных в строках. Сортировка словаря в Задании 4 сформировала упорядоченные коллекции по значениям в прямом и обратном направлениях, а для кортежа в Задании 6 определены минимальный (-1) и максимальный (15555) элементы.

На рисунке 2 приведены результаты функционирования функций и механизмов обработки исключений второй части.

● ● ● Jupyter Kernel - Выполнение Части 2 (Функции и исключения)

```
>>> # Задание 1: Проверка надежности пароля
Результат проверки 'PIUBIUBAFIUeB':
Проверка пароля завершена.
Результат: True
Завершение выполнения Задания 1

>>> # Задание 2: Функция diffTypeFunction с переменным типом
Кортеж: ('apple', 'banana', 'kiwi') -> len of apple is 5, len of banana is 6, len of kiwi is 4
Список: [1, 2, -3, 4, 5] -> sum after negative: 9
Число: 24681 -> amount of even digits: 4
Строка: 'some 10 20 words' -> sum of numbers in string: 30

>>> # Задание 3: Перестановка столбцов матрицы
input n and m: 2 3
Исходная матрица: [[1, 2, 3], [4, 5, 6]]
Операция перестановки столбцов 1 и 3 завершена.
Матрица после перестановки: [[3, 2, 1], [6, 5, 4]]

>>> # Задание 4: Демонстрация try/except/else/finally
Обработка '25' -> [else]: Успешный расчет, результат = 4.00 -> [finally]: Завершено
Обработка '0' -> [except ZeroDivisionError]: Деление на ноль -> [finally]: Завершено
Обработка 'abc' -> [except ValueError]: Ошибка преобразования -> [finally]: Завершено
```

Рисунок 2 – Результаты тестирования функций и обработки исключений части 2

Функция `is_password_good` для тестового значения «PIUBIUBAFIUeB» подтвердила выполнение всех трех критериев безопасности (длина ≥ 10 , наличие верхнего и нижнего регистра) с обязательным срабатыванием блока `finally`. В универсальной функции `diffTypeFunction` успешно обработаны четыре типа аргументов: для списка `[1, 2, -3, 4, 5]` сумма после первого отрицательного элемента составила $4 + 5 = 9$, для числа `24681` подсчитано 4 четных цифры, а для строки «some 10 20 words» сумма чисел составила 30. Операция перестановки 1-го и 3-го столбцов матрицы размером 2×3 корректно изменила порядок элементов в каждой строке. Комплексный тест блока `try..except..else..finally` подтвердил автономную изоляцию ошибок `ZeroDivisionError` и `ValueError` без аварийной остановки приложения.

На рисунке 3 проиллюстрировано пошаговое тестирование методов динамического списка `cheengizsList`.

● ● ● Jupyter Kernel - Выполнение Части 3. Задание 1 (Односвязный список)

```
>>> # Создание односвязного списка cheengizList
lst = cheengizList(10, 20, 30)
Исходное состояние: 10 -> 20 -> 30

>>> # Вызов метода append(40) (добавление в конец)
Результат: 10 -> 20 -> 30 -> 40

>>> # Вызов метода prepend(5) (добавление в начало)
Результат: 5 -> 10 -> 20 -> 30 -> 40

>>> # Вызов метода insert(2, 15) (вставка по индексу)
Результат: 5 -> 10 -> 15 -> 20 -> 30 -> 40

>>> # Вызов методов get(3) и find(30)
Элемент по индексу 3: 20
Индекс элемента со значением 30: 4

>>> # Вызов метода remove(20) (удаление элемента по значению)
Результат: 5 -> 10 -> 15 -> 30 -> 40

>>> # Вызов метода reverse() (обращение списка)
Результат после разворота: 40 -> 30 -> 15 -> 10 -> 5
Размер списка: 5 элементов
```

Рисунок 3 – Демонстрация вызова методов пользовательского класса односвязного списка

Были последовательно проверены все операции односвязного списка: создание из трех элементов ($10 \rightarrow 20 \rightarrow 30$), добавление в конец (`append(40)`), в начало (`prepend(5)`), вставка по индексу (`insert(2, 15)`), доступ по индексу (`get(3)` вернул 20), поиск позиции (`find(30)` вернул 4), удаление узла по значению (`remove(20)`) и разворот связей (`reverse()`), сформировавший инвертированный список из 5 узлов ($40 \rightarrow 30 \rightarrow 15 \rightarrow 10 \rightarrow 5$).

На рисунке 4 представлены результаты экономического и временного анализа поездки по маршруту Минск – Москва для иерархии классов транспортных средств.

● ● ● Jupyter Kernel - Выполнение Части 3. Задание 2 (Иерархия классов ТС)

```
>>> # Анализ поездок по маршруту Минск -> Москва (дистанция 720 км)

=====
Анализ поездок по маршруту Минск -> Москва (720 км)
=====

Самолет      : время = 0.9 ч,  стоимость = 8640.0 руб.
Поезд        : время = 7.2 ч,  стоимость = 2520.0 руб.
Автомобиль   : время = 8.0 ч,  стоимость = 3960.0 руб.

=====
Наиболее быстрая поездка      : Самолет (0.9 ч)
Наиболее экономичная поездка : Поезд (2520.0 руб.)
=====

>>> # Экспорт данных
Результаты успешно сохранены в файл: trip_report.txt
[OK] Файловый вывод завершен без ошибок.
```

Рисунок 4 – Сравнительные показатели транспортных средств по маршруту Минск – Москва

Для дистанции 720 км наиболее оперативным транспортом признан самолет (0,90 ч при стоимости 8640,0 руб.), а наиболее экономичным —

поезд (2520,0 руб. при времени в пути 7,20 ч). Сводный отчет был успешно сериализован в дисковый файл `trip_report.txt`.

На рисунке 5 отображена работа библиотечного каталога и выгрузка структуры в файл.

```
● ● ● Jupyter Kernel - Выполнение Части 3. Задание 3 (Каталог библиотеки)

>>> # Управление библиотечным фондом «Национальная библиотека»
Отдел 'Художественная литература' добавлен в библиотеку
Отдел 'Научная литература' добавлен в библиотеку
Книга «Война и мир» (Л. Толстой, 1869) добавлена в отдел 'Художественная литература'
Книга «1984» (Дж. Оруэлл, 1949) добавлена в отдел 'Художественная литература'
Книга «Краткая история времени» (С. Хокинг, 1988) добавлена в отдел 'Научная литература'

>>> # Модификация каталога
В отделе 'Художественная литература' название книги изменено с '1984' на '1984 (Специальное издание)'
Книга «Война и мир» удалена из отдела 'Художественная литература'

>>> # Сериализация структуры в файл
Данные библиотеки успешно записаны в файл: library_catalog.txt
[OK] Каталог синхронизирован с диском.
```

Рисунок 5 – Функционирование системы каталогизации книг библиотеки

В структуру библиотеки были добавлены два предметных отдела, произведено добавление литературы, модификация названия издания «1984» и исключение книги из каталога. Итоговая иерархическая структура была выгружена в файл `library_catalog.txt`.

6 ВЫВОДЫ

В ходе выполнения лабораторной работы № 1 получены следующие практические результаты:

1. Освоен базовый синтаксис языка Python, исследованы особенности динамической типизации и реализованы алгоритмы фильтрации, преобразования и сортировки стандартных коллекций (str, list, dict, tuple).
2. Изучена методология создания функций и построена надежная система обработки исключительных ситуаций (TypeError, ValueError, IndexError, ZeroDivisionError) на основе полной формы конструкции try...except...else...finally.
3. На практике закреплены принципы объектно-ориентированного программирования:
 - реализован класс односвязного динамического списка cheengizsList со всеми базовыми операциями модификации и обхода;
 - построена полиморфная иерархия классов транспортных средств с расчетом параметров рейсов;
 - реализована система каталогизации библиотечного фонда с сокрытием внутренних атрибутов через декораторы @property и файловым экспортом.
4. Проведено тестирование всех программных модулей, подтвердившее их полную работоспособность и соответствие требованиям индивидуального варианта № 7.